

[AFTERLEXX LOGO]

LIVE · INTERNAL DOCUMENTATION

Shahid Navalgund Trading Bot

Built by AFTERLEXX

A crypto paper-trading system that trades BTC, ETH, and SOL against real live prices with simulated money — rule-based strategies, a real backtest gate, half-Kelly sizing, and a periodic self re-vetting loop that promotes and demotes its own strategies over time.

Why I built this

I built this. It's a paper-trading bot for crypto — BTC, ETH, and SOL — that trades against real, live market prices with simulated money, using rule-based strategies that have to prove themselves on real historical data before they're allowed anywhere near a live decision. No strategy goes live on a hunch. No fill, price, or profit in this system is ever fabricated — if the bot can't get a real number, it says so and skips the cycle instead of making one up.

I built it with Claude (Anthropic) as my AI engineering partner — not as a black-box "AI trading brain" bolted onto the front, but as the tool I used to design, implement, test, and review every module in this codebase, task by task, with real test evidence at every step. This document is that system, written down completely enough that anyone on the AFTERLEXX team could stand up an identical instance from scratch.

— Shahid Navalgund, AFTERLEXX

TABLE OF CONTENTS

01 Overview	tech stack, scope
02 Architecture	system diagram
03 Core Build Principles	the real rules behind every module
04 System Components	data · signal · risk · execution · re-vetting · dashboard
05 Operational Guide	the 11-step live cycle
06 Setup & Configuration	keys, env, dependencies
07 Rebuilding From Scratch	build order
08 Disclaimer	internal use, risk notice

01 Overview

Runs a continuous paper-trading loop against three cryptocurrencies, using real live prices, real (simulated) fees and slippage, and rule-based strategies that are only allowed to trade if they pass a real backtest first. Every few hours it re-checks its own strategies against fresh data and promotes or demotes them automatically — that's the "learns from its mistakes" behavior. It does not execute real trades, hold real funds, or connect to any exchange account.

Assets traded	BTC/USDT, ETH/USDT, SOL/USDT (Binance spot pairs)
Mode	Paper trading — simulated cash, real prices
Starting balance	\$10,000.00 (simulated)
Data source	Binance public REST API — no API key required
Language / runtime	Python 3.9
Persistence	SQLite — single local .db file
Dashboard	Streamlit + Altair
Strategies	SMA crossover, RSI threshold, Momentum — rule-based, no runtime LLM inference
Position sizing	Half-Kelly, from each strategy's real backtested win rate & payoff ratio
Poll interval	Every 300 seconds (5 minutes)
Re-vet interval	Every 72 cycles (~6 hours)
AI's role	Development partner — design, implementation, testing, review. Not a runtime trading component.

02 Architecture

Two independent OS processes share one SQLite file: the background loop (writer) and the dashboard (reader, plus one live price call of its own for unrealized P&L).



03 Core Build Principles

The actual operating rules I held Claude to throughout development — the real instructions that shaped every module in Section 04, not illustrative examples. Every commit in this codebase was built and reviewed against these.

`RULE – Never fake a fill, a price, or a profit`

Applies to every module that touches money or market data. If a real price/fill/fee can't be obtained, the system says so and skips that unit of work — never substitutes a guessed or last-known-good value and presents it as current.

`RULE – Verify, don't guess`

Every constant, formula, and API assumption was checked against real API responses or official docs before being written into code. Where a value genuinely couldn't be verified, it's explicitly disclosed as an assumption, not presented as fact.

`RULE – A strategy only trades live if it earns it`

No strategy is wired into the live loop directly. It must pass a real backtest against real historical candles first, and it's re-checked periodically — passing once is not permanent.

`RULE – Every task ships with real test evidence`

Each module was built test-first, with actual pytest output attached to every commit, plus an independent code review pass before being marked done.

04 System Components

4.1 — Market Data Layer `tradingbot/data/binance_feed.py`

Fetches real live prices and historical candles from Binance's public spot API. No API key required.

MARKET DATA FETCH

```
BASE_URL = "https://api.binance.com/api/v3"

def get_price(symbol):
    # GET {BASE_URL}/ticker/price?symbol={symbol}
    # Raises PriceFetchError on non-200 - never returns a stale price.

def get_klines(symbol, interval, limit):
    # GET {BASE_URL}/klines?symbol=&interval=&limit=
    # Returns [{open_time, open, high, low, close, volume}, ...]
```

4.2 — Signal Layer `tradingbot/strategies/`

Three independent, deterministic, rule-based strategies. Each implements `signal(closes) -> "buy" | "sell" | "hold"`.

SMA CROSSOVER — FAST=10, SLOW=30

```
if fast_prev ≤ slow_prev and fast_now > slow_now: return "buy"
if fast_prev ≥ slow_prev and fast_now < slow_now: return "sell"
```

RSI THRESHOLD — PERIOD=14, OVERSOLD=30, OVERBOUGHT=70

```
if rsi_prev ≤ oversold and rsi_now > oversold: return "buy"
if rsi_prev ≥ overbought and rsi_now < overbought: return "sell"
```

MOMENTUM — LOOKBACK=20, THRESHOLD=0.0

```
ret = (closes[-1] - closes[-(lookback+1)]) / closes[-(lookback+1)]
if ret > threshold: return "buy"
if ret < -threshold: return "sell"
```

4.3 — Risk Layer tradingbot/backtest/ + tradingbot/sizing/

Backtest gate — a strategy earns the right to trade live:

```
BACKTEST PASS GATE  
FEE_RATE = 0.001 // 0.1%, same as live taker fee  
  
passed = sharpe > 0 and max_drawdown < 0.5 and num_trades ≥ 2
```

Position sizing — half-Kelly, from the strategy's own real backtested trade outcomes:

```
HALF-KELLY POSITION SIZING  
  
def kelly_fraction(win_rate, payoff_ratio):  
    p, q, b = win_rate, 1 - win_rate, payoff_ratio  
    if b ≤ 0: return 0.0  
    return max(0.0, min((b*p - q) / b, 1.0))  
  
def position_size(equity, win_rate, payoff_ratio, fraction=0.5):  
    return equity * kelly_fraction(win_rate, payoff_ratio) * fraction
```

4.4 — Execution Layer tradingbot/engine/broker.py + portfolio.py

Applies real, honest transaction costs to every simulated fill:

```
FEES & SLIPPAGE  
  
TAKER_FEE_RATE = 0.001 // 0.1% — matches real Binance fee schedule  
SLIPPAGE_RATE = 0.0005 // 0.05% fixed unfavorable slippage, every trade  
  
// buy fills above market; sell fills below — always unfavorable
```

Portfolio raises real errors (`InsufficientFundsError`, `InsufficientPositionError`) rather than silently allowing an impossible trade.

4.5 — Re-vetting Layer tradingbot/loop/main.py

The "learns from its mistakes" behavior. Every 72 cycles (~6 hours), every strategy is re-backtested against fresh 500-candle hourly history:

```
PERIODIC RE-VETTING
REJET_INTERVAL_CYCLES = 72 // 6 hours at the 300s poll interval

def maybe_revet(conn, cycles_since_vet, approved_strategies, ...):
    cycles_since_vet += 1
    if cycles_since_vet < revet_interval:
        return approved_strategies, cycles_since_vet
    try:
        new_approved, _ = vet_fn(conn)
    except (PriceFetchError, RequestException):
        // keep current approvals, retry next interval - never crash
        return approved_strategies, 0
    return new_approved, 0
```

A strategy that starts failing gets demoted — blocked from new positions, but still allowed to close any position it already holds. A previously-rejected strategy that starts passing gets promoted.

4.6 — Dashboard tradingbot/dashboard/app.py

Streamlit + Altair, reads directly from SQLite plus one live price call for unrealized P&L. Dark terminal theme, wallet breakdown, per-coin cards with 24h sparklines, color-coded open-position cards, gradient equity curve, trade log, and honest strategy vetting history.

05 Step-by-Step Operational Guide

The full cycle, exactly as `tradingbot/loop/main.py` executes it every 300 seconds.

- 1 **Startup vetting** — on process start, fetch 500 hourly candles per symbol, backtest all 3 strategies against each, log every result to `strategy_runs`, build the initial approved list.
- 2 **Seed price history** — reuse the same 500-candle series as the starting live signal history.
- 3 **Poll live prices** — fetch current price per symbol. On failure, skip that symbol this cycle only — never substitute a stale price.
- 4 **Append to price history** — each successful price grows that symbol's running series.
- 5 **Build the evaluate-set** — currently-approved combos, plus any demoted combo that still has an open position.
- 6 **Evaluate signals** — call `strategy.signal(closes)` for each combo in the evaluate-set.
- 7 **Act on buy signals** — if approved and no position is open: size via half-Kelly, open the trade.
- 8 **Act on sell signals** — if a position is open: close it, regardless of current approval status.
- 9 **Log equity** — compute total portfolio equity, insert one row into `equity_history`.
- 10 **Re-vet check** — every 72 cycles, re-run vetting and replace the approved list (errors keep current approvals, never crash).
- 11 **Sleep** — wait 300 seconds, repeat from step 3.

Any unexpected exception in a single cycle is caught at the top level and logged — it never kills the loop process.

06 Setup & Configuration

API keys required: none. Binance's spot ticker/price and klines endpoints are public and unauthenticated. No `.env` file, no secrets, by design.

Constant	Value	Meaning
SYMBOLS	["BTCUSDT","ETHUSDT","SOLUSDT"]	Traded pairs
POLL_SECONDS	300	Live poll interval
RENET_INTERVAL_CYCLES	72	Re-vetting interval (~6 hrs)
STARTING_CASH	10000.0	Simulated starting balance
DB_PATH	"tradingbot.db"	SQLite file location

REQUIREMENTS.TXT

```
requests ≥ 2.31
pandas ≥ 2.2
numpy ≥ 1.26
streamlit ≥ 1.35
pytest ≥ 8.0
// altair installs transitively via streamlit – pin explicitly if you ever swap it out
```

RUN IT

```
pip install -r requirements.txt

python -m tradingbot.loop.main          // background trading loop
streamlit run tradingbot/dashboard/app.py // dashboard, separate terminal
```

07 Rebuilding From Scratch

- 1 Set up the four core packages: `data/` (market data), `strategies/` (signal generation), `backtest/` (pass/fail gate), `sizing/` (Kelly position sizing).
- 2 Build `engine/` (portfolio state, broker fill logic, SQLite schema/CRUD) on top — the only layer that mutates money state.
- 3 Wire it together in `loop/main.py` following the 11-step cycle in Section 05 exactly — vet once at startup, then poll / evaluate / act / log / sleep, with periodic re-vetting layered on top.
- 4 Build `dashboard/app.py` as a read-only consumer of the same SQLite file, plus its own single live price call for unrealized P&L display.
- 5 At every step, hold to Section 03's four rules — especially "never fake a fill, a price, or a profit." That rule is what separates this from a toy backtest dressed up as a live system.

08 Disclaimer

This system is for internal AFTERLEXX use only. It trades with simulated money against real market data — it does not execute real trades, hold real funds, or connect to any exchange account. Nothing in this document or the underlying codebase constitutes financial advice. Trading cryptocurrency involves substantial risk of loss; past backtest performance does not predict future results. Do not represent this system's output as investment advice to any third party.

Shahid Navalgund Trading Bot — Built by AFTERLEXX
real prices, fake money, and it never lies.